LAB 9 / DESIGN ANALYSIS REPORT

Digital Health Management System

System analysis, logical design, data model, workflows, and quality considerations

Name: _____	**Roll No.:** _____
Course / Section: _____	**Date:** _____

1. Aim

To analyse the requirements and logical design of the Digital Health Management System (DHMS), identify its users, modules, data entities, interfaces, workflows, constraints, and quality requirements, and evaluate how the design supports secure and coordinated healthcare information management.

2. Project Overview

DHMS is a proposed role-based digital platform for centralising patient information and common healthcare workflows. It connects patient registration, appointment scheduling, clinical encounters, prescriptions, laboratory and diagnostic reports, notifications, follow-up tasks, administration, and audit records. Patients and authorised staff use a responsive web or mobile-browser interface. The system communicates with external services only through controlled interfaces.

3. Problem Analysis

In a fragmented or paper-based workflow, appointments, patient history, test reports, and follow-up actions may be stored in separate registers, spreadsheets, calls, or departmental applications. This creates repeated data entry, unclear booking status, slow record retrieval, delayed report review, and dependence on staff memory for follow-up. The design response is a shared information-management platform with controlled access, linked records, explicit status tracking, and an audit trail.

4. Objectives

- Centralise patient demographics and longitudinal health records.
- Support appointment creation, rescheduling, cancellation, check-in, and status tracking.
- Allow authorised clinicians to document encounters, diagnoses, care plans, and prescriptions.
- Store laboratory orders and reports with status, source, and provenance information.
- Coordinate notifications and follow-up tasks while respecting user preferences.
- Protect health information through role-based permissions, encryption, and auditability.

5. Scope and System Boundary

Within the proposed system	Outside the system boundary / not claimed
Patient accounts and profiles; appointments; encounter documentation; prescriptions; laboratory report metadata and files; notifications; follow-up tasks; administration; audit and operational reporting.	Autonomous diagnosis, replacement of clinician judgement, emergency response, and guaranteed integration with every hospital, laboratory, pharmacy, or payment provider.
Optional interfaces for identity, email/SMS/push, laboratory/FHIR exchange, and payment/billing where configured.	Specific third-party products, deployment hosting, production legal retention rules, and final role-permission matrix remain implementation decisions.

6. Stakeholder and User Analysis

The system applies role-based access control. A user should see and modify only the information required for the tasks permitted to that role. The final permission matrix must be confirmed by the organisation before deployment.

User class	Primary responsibilities	Access focus
Patient	Manage own profile, appointments, consent/preferences, and view information shared with the patient.	Own records, appointments, prescriptions and permitted finalized reports.
Clinician	Review patient history; conduct encounters; enter notes, diagnoses, care plans, and prescriptions.	Assigned patient records and clinical workflows.
Receptionist	Register patients, coordinate provider schedules, check patients in/out, manage appointment queues.	Demographics and operational appointment data; no unrestricted clinical editing.
Pharmacist / Medication Staff	Review prescriptions and update medication fulfilment-related status where included.	Medication workflow within assigned permissions.
Laboratory Staff	Create or receive test orders, upload and verify diagnostic reports.	Lab orders, result metadata, report upload and verification.
Administrator	Manage accounts, roles, departments, providers, configuration, and authorised audit/report review.	System configuration and operational reporting; clinical data access is not automatic.

7. Functional Analysis and Major Modules

Module	Main design responsibility	Key output
1. Account & access management	Authentication, recovery, sessions, multi-factor authentication support, and server-side RBAC.	Authenticated session and permission decision.
2. Patient profile & health records	Maintain demographics, contacts, allergies, conditions, medications, and attachments.	Authorised patient record view.
3. Appointment management	Find slots, create/reschedule/cancel bookings, prevent conflicts, and track status.	Appointment and status history.
4. Clinical documentation	Create encounters, observations, notes, diagnoses, care plans, and amendments.	Draft or finalized encounter record.
5. Prescription management	Create prescriptions, validate dosage fields, display medication history and allergy information.	Prescription with medicine instructions.
6. Lab & diagnostic reports	Manage lab orders, upload results, validate files, and track review status/provenance.	Report metadata and securely stored document.
7. Notifications & follow-up	Schedule reminders, respect preferences, create follow-up tasks, and record delivery status.	Notification / follow-up status.
8. Administration, audit & reporting	Manage configuration and roles; record sensitive events; provide role-appropriate dashboards.	Audit events and operational reports.

8. Main Use-Case Summary

A patient authenticates, checks availability, selects a provider and time slot, and submits a booking. The system rechecks availability before creating the appointment, records the booking, and queues a confirmation notification. On the visit date, authorised staff check the patient in. The clinician opens the patient context, records the encounter, and may create a prescription or laboratory order. Laboratory staff attach results; the report remains in the appropriate review state until an authorised clinician reviews it. The system records important actions and creates follow-up tasks or reminders as configured.

9. Architectural Design Analysis

The logical architecture separates user interaction, business rules, persistent data, and external integrations. This reduces coupling, supports independent testing of modules, and gives the system a controlled boundary for sensitive health information.

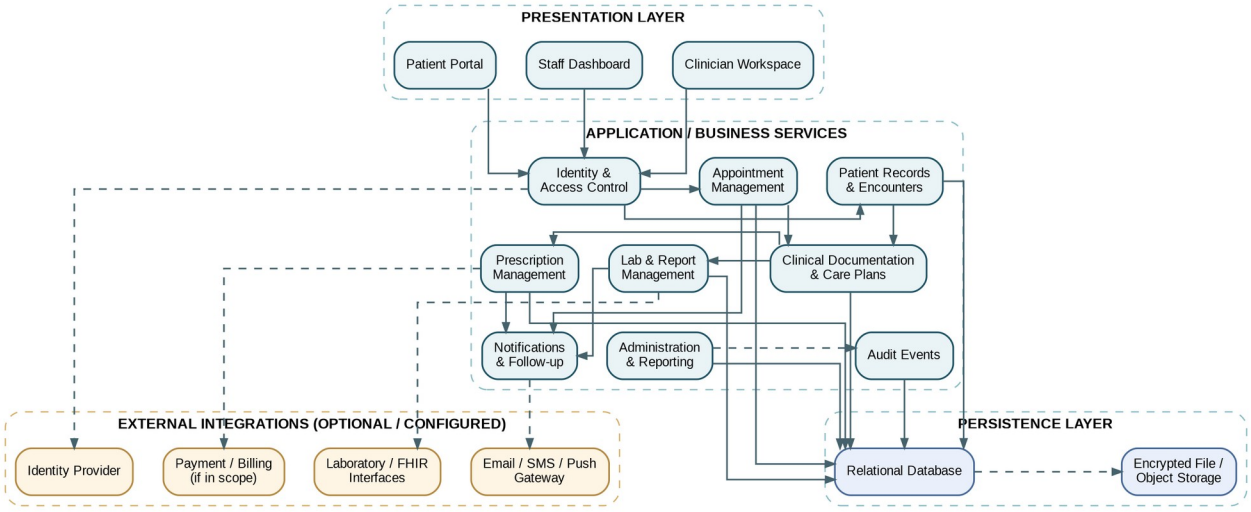


Figure 1. Logical layered architecture and principal integrations of DHMS.

Layer	Components / responsibility	Design analysis
Presentation	Patient portal, staff dashboard, clinician workspace.	Role-specific, responsive screens; validation and clear status feedback; the interface is not the security boundary.
Application / business services	Identity and access, appointments, patient records, clinical documentation, prescriptions, laboratory reports, notifications, administration, audit.	Business rules live in services and are enforced on the server. Modules communicate through explicit APIs.
Persistence	Relational database and encrypted file/object storage.	Structured data and report metadata are stored consistently; large attachments are protected in file storage.
Integration	Identity provider, messaging gateway, lab/FHIR interfaces, optional billing/payment gateway.	External dependencies are isolated behind authenticated interfaces with timeout, retry, and duplicate-prevention behavior.

10. Design Rationale

- Modularity: appointment, clinical, prescription, laboratory, and notification concerns are separated so that each module has a clear responsibility.
- Least privilege: protected operations check the authenticated user and their server-side permissions, not merely which buttons are visible in the interface.
- Controlled integration: external systems are accessed through APIs or gateways instead of direct access to the internal database.
- Consistency: booking availability is rechecked at submission time to avoid two users claiming the same exclusive provider/resource slot.
- Auditability: authentication, sensitive record access, clinical changes, report access, and administrative changes produce audit events.

11. Data and Class Design Analysis

The conceptual data model separates account identity, patient identity, appointments, clinical encounters, prescriptions, laboratory workflow, notifications, and audit events. The diagram shows the principal entities and their relationships. Supporting classes such as PrescriptionItem and LabOrder are included to make individual medicine instructions and test workflows explicit.

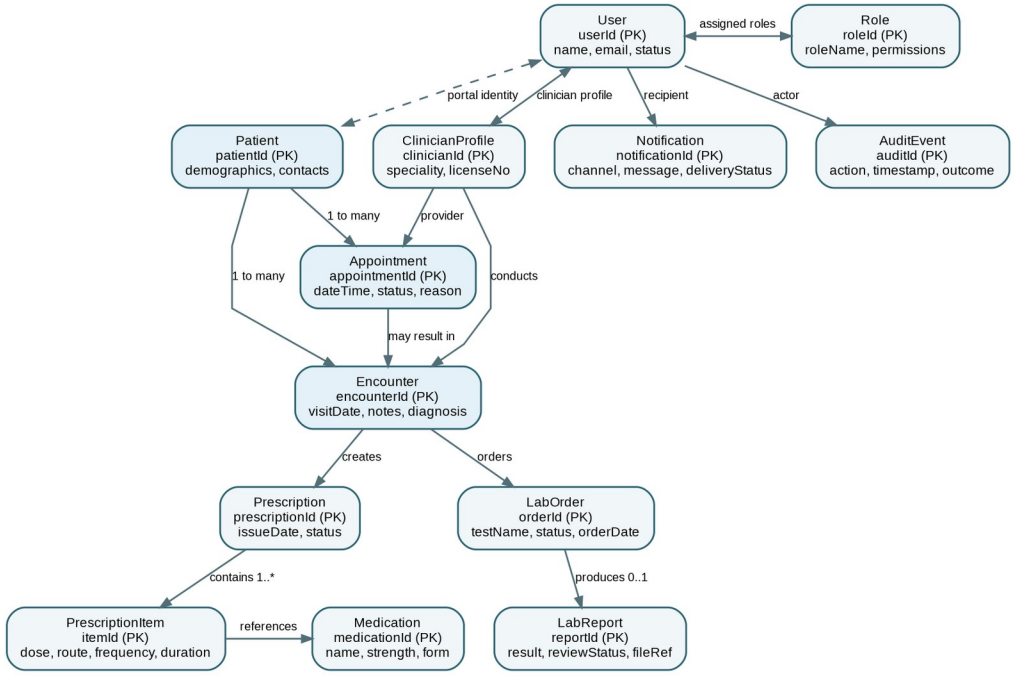


Figure 2. High-level conceptual data model. PK denotes a primary key; arrows indicate associations or lifecycle relationships.

12. Key Entity Responsibilities

Entity / class	Purpose and important data
User / Role	Account identity, contact/login details, account status, assigned roles and permissions.
Patient	Patient identifier, demographics, emergency contact, allergies, conditions, medication history and documents.
Clinician Profile	Clinician identifier, speciality and professional registration details linked to a user account.
Appointment	Patient, provider/resource, date/time, reason, booking status and status-change history.
Encounter	Patient, clinician, appointment/visit context, observations, diagnosis, notes and care plan.
Prescription / PrescriptionItem / Medication	Prescription header and status; each medicine entry stores dose, route, frequency, duration and instructions; medication catalogue stores medicine details.
LabOrder / LabReport	Test order, order date and status; report result, file reference, review status, source and provenance.
Notification / AuditEvent	Recipient, channel, message and delivery status; actor, action, target resource, timestamp and outcome.

13. Relationship and Lifecycle Findings

- A patient may have multiple appointments and encounters over time.
- An appointment may result in an encounter; walk-in encounters are supported where the workflow permits.
- An encounter may generate prescriptions and lab orders. Each prescription contains one or more medicine items linked to a medication.
- A lab order may produce a report; access depends on report status and permissions. Notifications link a recipient to an event, while audit events record actor, target, time, and outcome.

14. Process and State Analysis

14.1 Appointment lifecycle

The appointment process is modelled with explicit states so that every transition has a trigger and an authorised actor. The principal states are Scheduled, Confirmed, Checked-in, In-consultation, Completed, Cancelled, and No-show. Cancellation and no-show are alternative outcomes; they are not steps that every appointment must pass through.

Transition	Trigger / responsible actor	System response
Request to Scheduled / Confirmed	Patient or authorised staff submits a valid booking; system confirms slot availability.	Create unique appointment; store provider, time, and status; queue confirmation.
Scheduled to Confirmed	Booking rules and confirmation requirements are satisfied.	Record confirmation and notify relevant users.
Confirmed to Checked-in	Patient arrives; receptionist or authorised staff checks in.	Update appointment status and record actor/time.
Checked-in to In-consultation	Clinician begins the visit.	Open encounter context and record transition.
In-consultation to Completed	Clinician completes and saves the visit according to workflow.	Store encounter outcome and link prescriptions/reports/follow-up as needed.
Any permitted active state to Cancelled	Patient or authorised staff cancels under configured rules.	Record cancellation reason, actor/time, release slot and notify users.
Confirmed to No-show	Attendance cutoff passes without check-in.	Mark no-show according to clinic policy and record the event.

14.2 Important business rules

- The system shall prevent double booking of an exclusive provider/resource slot unless explicitly configured otherwise.
- Only authorised users may create, reschedule, cancel, or update appointment status.
- Clinical notes are separated into draft and finalized states; finalized records require controlled amendment.
- Laboratory reports must retain source and upload/finalization metadata and are exposed only to authorised users or patients where policy permits.
- Notification preferences are respected except for mandatory operational/security notices defined by policy.

15. Non-Functional and Security Analysis

Quality area	Design requirement / analysis
Performance	Standard authenticated page/API requests target a successful response within 2 seconds under nominal load; 95% of interactive requests should complete within 4 seconds excluding third-party delays.
Security & privacy	Use HTTPS/TLS for protected communications, encryption for stored sensitive data, secure session controls, server-side RBAC, least privilege, and controlled file access.
Auditability	Record authentication, patient-record access, sensitive clinical changes, report access, role changes, and administrative configuration changes.
Reliability & recovery	Use validated transactions, backup of critical data, periodic restore tests, and safe handling of temporary non-critical integration failures.
Usability & accessibility	Provide consistent role-specific workflows, clear status/error messages, keyboard and touch support, readable contrast, and text equivalents for icons.
Maintainability	Use modular services, explicit interfaces, automated tests, traceable requirements, and documented deployment configuration.

16. Assumptions, Constraints, and Dependencies

16.1 Assumptions and dependencies

- Users have access to a supported desktop or mobile browser and a secure network connection for normal operation.
- An organisation will define the final role/permission matrix, consent rules, record-retention periods, and disclosure workflow.
- Identity, messaging, laboratory, FHIR, and payment services are available only when selected, configured, and supported by the deployment organisation.
- External services may fail or be delayed; the core system must not corrupt already-finalized local records during non-critical integration outages.

16.2 Design constraints

- Health information is confidential and must not leak through client-side logs, debug output, analytics payloads, or unsecured URLs.
- Protected operations must be authorised on the server side, not solely by interface visibility.
- Clinical content must distinguish clinician-authored facts and actions from calculated or system-generated information.
- Technology stack, hosting platform, final integration versions, authentication provider, and jurisdiction-specific compliance requirements are not fixed in the current baseline.

17. Risk Analysis and Mitigation

Risk	Potential impact	Design mitigation
Unauthorised access	Exposure or modification of sensitive patient information.	Server-side RBAC, least privilege, strong authentication, session controls, audit review.
Duplicate/conflicting appointments	Double booking and disruption of clinical schedules.	Atomic availability recheck, conflict response, idempotent create operations and tests.
Incorrect or incomplete clinical data	Poor continuity of care or misinterpretation.	Field validation, author/time metadata, draft/final status, amendment workflow and clinician review.
External integration outage	Delayed notifications or lab/payment exchange.	Timeout/retry policy, queued delivery, visible failure status, and safe degradation.
Loss or corruption of records	Operational disruption and loss of history.	Automated backups, restoration tests, recovery runbook, and integrity checks.
Misleading automation	Users could mistake workflow assistance for diagnosis.	Explicitly keep clinical decisions with qualified healthcare professionals; DHMS does not autonomously diagnose.

18. Validation and Testing Strategy

Design validation should include unit tests for service rules, integration tests for database and external gateways, end-to-end tests for patient and staff journeys, role-based access tests, usability review, performance testing under agreed nominal load, and backup/restore exercises. Acceptance examples include blocking an overlapping appointment, denying an unrelated patient-record request and logging the denial, requiring an authorised amendment to a signed note, and keeping an uploaded lab result pending until clinician review is recorded.

19. Traceability to Earlier Lab Work

Lab	Model / artefact	How it contributes to this analysis
Lab 3	Class diagram	Identifies key domain objects and their dependencies.
Lab 4	Activity diagram	Shows end-to-end actions, decisions, and alternate paths.
Lab 5	Data Flow Diagram	Shows information movement between actors, processes, data stores, and external systems.
Lab 6	Sequence diagram	Shows time-ordered interactions for booking and consultation.
Lab 7	State transition diagram	Defines valid appointment states and the events that change them.
Lab 8	Component diagram	Shows software modules, interfaces, and architectural dependencies.

20. Conclusion

The design analysis indicates that DHMS should be built as a modular, role-based information-management platform with linked patient, appointment, encounter, prescription, and laboratory records. The architecture separates presentation, application services, persistence, and external integrations. Explicit workflows, conflict checks, record-finalisation rules, audit events, and privacy controls address the main operational risks. Before implementation or deployment, the project team must confirm the final permissions, technology stack, integration contracts, retention policy, and applicable legal requirements. Clinical decisions remain the responsibility of qualified healthcare professionals.

Final design takeaway A dependable healthcare management system is not just a collection of forms. It is a controlled set of linked records, authorised workflows, verifiable rules, and accountable actions.